Lecture # 8-9 More SQL – Complex Queries, Triggers, Views

Rashmi Dutta Baruah

Department of Computer Science & Engineering

भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
Indian Institute of Technology Guwahati
Guwahati - 781039, INDIA

Outline

- Complex SQL Retrieval Queries
- Specifying constraints as assertions and actions as triggers.
- Views (virtual tables) in SQL

NULLS in SQL Queries

- NULL is used to represent missing value with three different interpretations:
  - Unknown value (a person's DoB is not known)
  - Unavailable or withheld value (a person has a home phone number but does not want it to be listed)
  - Not applicable attribute (college_degree attribute would be NULL for a person who has no college degrees)

- Not possible to determine which of the meanings is intended.

- SQL uses a three-value logic with values TRUE, FALSE, and UNKNOWN.

THREE-VALUED LOGIC

Table 5.1 Logical Connectives in Three-Valued Logic

(a)

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

(b)

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

(c)

NOT	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

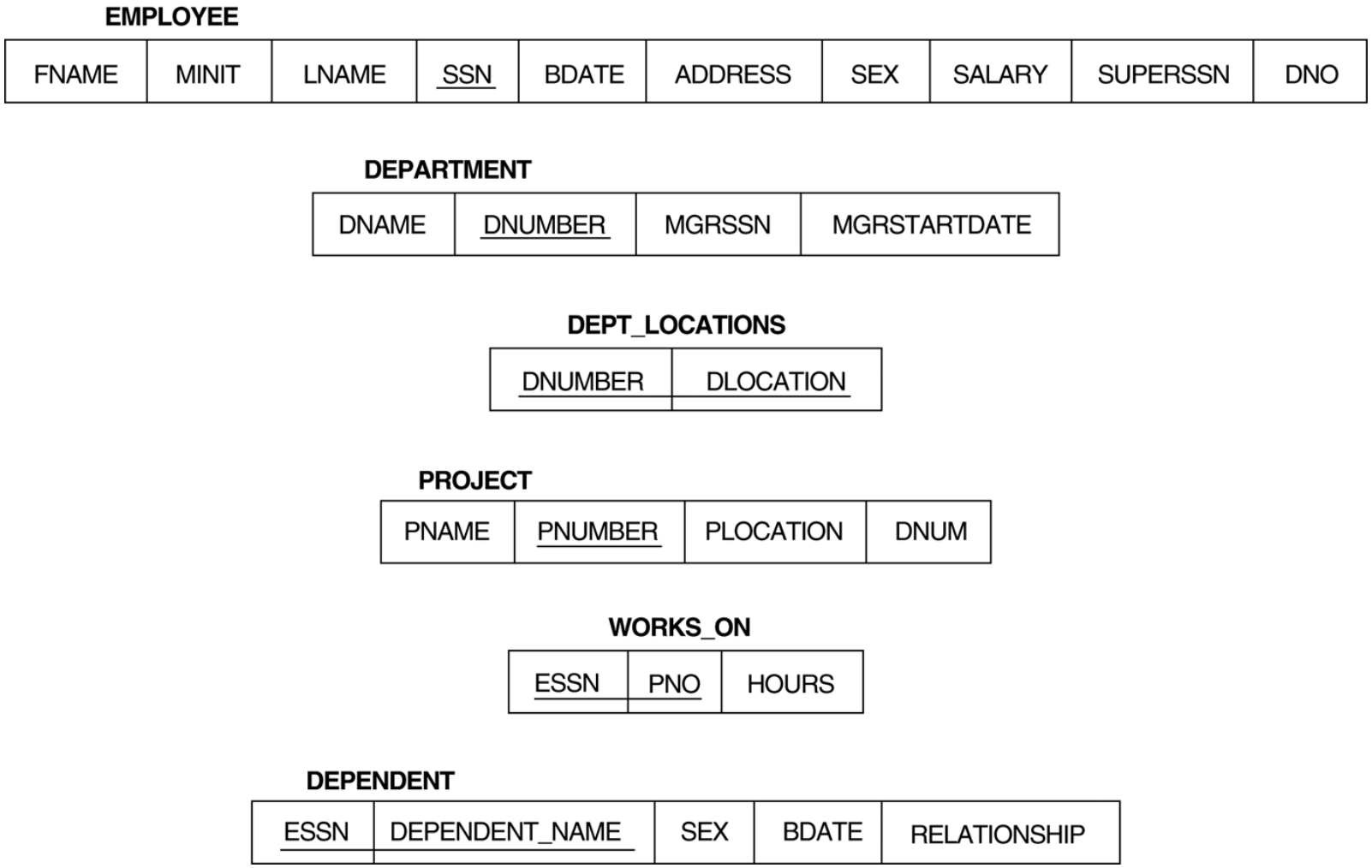
- Similarly, any operation involving an unknown value produces an unknown value for the result.
  - e.g., *unknown* + 5 → *unknown*

NULLS IN SQL QUERIES

- SQL allows queries that check if a value is NULL (missing or undefined or not applicable)
- SQL uses **IS** or **IS NOT** to compare NULLs
- Query 18: Retrieve the names of all employees who do not have supervisors.

  Note: If a join condition is specified, tuples with NULL values for the join attributes are not included in the result

Relational Database Schema



EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

Populated Database

NULLS IN SQL QUERIES

- Query 18: Retrieve the names of all employees who do not have supervisors.

```
Q18:  SELECT    FNAME, LNAME
      FROM      EMPLOYEE
      WHERE     SUPERSSN  IS  NULL
```

NESTING OF QUERIES

- A complete SELECT query, called a *nested query* , can be specified within the WHERE-clause of another query, called the *outer query*

- Many of the previous queries can be specified in an alternative form using nesting

- Query 1: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1:     SELECT  FNAME, LNAME, ADDRESS
        FROM    EMPLOYEE
        WHERE   DNO IN  (SELECT  DNUMBER
                         FROM    DEPARTMENT
                         WHERE   DNAME='Research' )
```

NESTING OF QUERIES (cont.)

- The nested query selects the number of the 'Research' department
- The outer query select an EMPLOYEE tuple if its DNO value is in the result of nested query
- The comparison operator **IN** compares a value v with a set (or multi-set) of values V, and evaluates to **TRUE** if v is one of the elements in V
- In general, we can have several levels of nested queries
- A reference to an *unqualified attribute* refers to the relation declared in the *innermost nested query*

CORRELATED NESTED QUERIES

- If a condition in the WHERE-clause of a *nested query* references an attribute of a relation declared in the *outer query* , the two queries are said to be *correlated*
- The result of a correlated nested query is *different for each tuple (or combination of tuples) of the relation(s) the outer query*
- Query 12: Retrieve the name of each employee who has a dependent with the same first name as the employee.

```
Q12: SELECT   E.FNAME, E.LNAME
     FROM     EMPLOYEE AS E
     WHERE    E.SSN IN (SELECT ESSN
                        FROM   DEPENDENT
                        WHERE  ESSN=E.SSN AND
                               E.FNAME=DEPENDENT_NAME)
```

CORRELATED NESTED QUERIES (cont.)

- In Q12, the nested query has a different result *for each tuple* in the outer query
- A query written with nested SELECT... FROM... WHERE... blocks and using the = or IN comparison operators can ***always*** be expressed as a single block query. For example, Q12 may be written as in Q12A

```
Q12A:   SELECT  E.FNAME, E.LNAME
        FROM    EMPLOYEE E, DEPENDENT D
        WHERE E.SSN=D.ESSN AND
                E.FNAME=D.DEPENDENT_NAME
```

THE EXISTS FUNCTION

- EXISTS is used to check whether the result of a correlated nested query is empty (contains no tuples) or not
- We can formulate Query 12 in an alternative form that uses EXISTS as Q12B below

Query 12: Retrieve the name of each employee who has a dependent with the same first name as the employee.

```
Q12B:   SELECT  FNAME, LNAME
        FROM    EMPLOYEE
        WHERE   EXISTS  (SELECT  *
                FROM    DEPENDENT
                WHERE   SSN=ESSN AND
                        FNAME=DEPENDENT_NAME)
```

THE NOT EXISTS FUNCTION

- Query 6: Retrieve the names of employees who have no dependents.

```
Q6:     SELECT  FNAME, LNAME
        FROM    EMPLOYEE
        WHERE   NOT EXISTS  (SELECT     *
                            FROM  DEPENDENT
                            WHERE SSN=ESSN)
```

- In Q6, the correlated nested query retrieves all DEPENDENT tuples related to an EMPLOYEE tuple. If *none exist* , the EMPLOYEE tuple is selected

- EXISTS is necessary for the expressive power of SQL

EXPLICIT SETS

- It is also possible to use an **explicit (enumerated) set of values** in the WHERE-clause rather than a nested query

- Query 13: Retrieve the social security numbers of all employees who work on project number 1, 2, or 3.

```
Q13:    SELECT   DISTINCT ESSN
        FROM     WORKS_ON
        WHERE    PNO IN  (1, 2, 3)
```

Joined Tables in SQL

- Can specify a “joined table” or "joined relation" in the FROM-clause
- Looks like any other relation but is the result of a join
- Allows the user to specify different types of joins (regular "theta" JOIN, NATURAL JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN, etc)

Joined Tables in SQL and Outer Joins

- **Q1: SELECT FNAME, LNAME, ADDRESS**
  FROM EMPLOYEE, DEPARTMENT
  WHERE DNAME='Research' AND DNUMBER=DNO

- could be written as:

  Q1: SELECT FNAME, LNAME, ADDRESS
  FROM (EMPLOYEE JOIN DEPARTMENT
  ON DNUMBER=DNO)
  WHERE DNAME='Research'

  or as:

  Q1: SELECT FNAME, LNAME, ADDRESS
  FROM (EMPLOYEE NATURAL JOIN DEPARTMENT
  AS DEPT(DNAME, DNO, MSSN, MSDATE))
  WHERE DNAME='Research'

-

AGGREGATE FUNCTIONS

- Include **COUNT**, **SUM**, **MAX**, **MIN**, and **AVG**
- Query 15: Find the maximum salary, the minimum salary, and the average salary among all employees.

```
Q15: SELECT   MAX(SALARY),
        MIN(SALARY), AVG(SALARY)
                FROM    EMPLOYEE
```

  - Some SQL implementations *may not allow more than one function* in the SELECT-clause

AGGREGATE FUNCTIONS (cont.)

- Query 16: Find the maximum salary, the minimum salary, and the average salary among employees who work for the 'Research' department.

```
Q16: SELECT     MAX(SALARY), MIN(SALARY),
                AVG(SALARY)
     FROM  EMPLOYEE, DEPARTMENT
     WHERE      DNO=DNUMBER AND
                DNAME='Research'
```

AGGREGATE FUNCTIONS (cont.)

- Queries 17 and 18: Retrieve the total number of employees in the company (Q17), and the number of employees in the 'Research' department (Q18).

```
Q17: SELECT   COUNT (*)
              FROM    EMPLOYEE

Q18: SELECT   COUNT (*)
              FROM    EMPLOYEE, DEPARTMENT
              WHERE   DNO=DNUMBER AND DNAME='Research'
```

GROUPING

- In many cases, we want to apply the aggregate functions *to subgroups of tuples in a relation*
- Each subgroup of tuples consists of the set of tuples that have *the same value* for the *grouping attribute(s)*
- The function is applied to each subgroup independently
- SQL has a **GROUP BY**-clause for specifying the grouping attributes, which *must also appear in the SELECT-clause*

GROUPING (cont.)

- Query 20: For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
Q20: SELECT     DNO, COUNT (*), AVG (SALARY)
     FROM       EMPLOYEE
     GROUP BY   DNO
```

  - In Q20, the EMPLOYEE tuples are divided into groups--each group having the same value for the grouping attribute DNO
  - The COUNT and AVG functions are applied to each such group of tuples separately
  - The SELECT-clause includes only the grouping attribute and the functions to be applied on each group of tuples
  - A join condition can be used in conjunction with grouping

GROUPING (cont.)

- Query 21: For each project, retrieve the project number, project name, and the number of employees who work on that project.

```
Q21:    SELECT      PNUMBER, PNAME, COUNT (*)
        FROM        PROJECT, WORKS_ON
        WHERE       PNUMBER=PNO
        GROUP BY    PNUMBER, PNAME
```

  - In this case, the grouping and functions are applied *after* the joining of the two relations

THE HAVING-CLAUSE

- Sometimes we want to retrieve the values of these functions for only those *groups that satisfy certain conditions*

- The HAVING-clause is used for specifying a selection condition on groups (rather than on individual tuples)

- Query 22: For each project *on which more than two employees work* , retrieve the project number, project name, and the number of employees who work on that project.

```
Q22:    SELECT    PNUMBER, PNAME, COUNT (*)
        FROM      PROJECT, WORKS_ON
        WHERE     PNUMBER=PNO
        GROUP BY  PNUMBER, PNAME
        HAVING    COUNT (*) > 2
```

Constraints as Assertions in SQL

- General constraints: constraints that do not fit in the basic SQL categories

- Mechanism: `CREAT ASSERTION`
  - components include: a constraint name, followed by `CHECK`, followed by a condition

Assertions: An Example

- "The salary of an employee must not be greater than the salary of the manager of the department that the employee works for''

```
CREAT ASSERTION SALARY_CONSTRAINT
CHECK (NOT EXISTS (SELECT *
 FROM EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
 WHERE E.SALARY > M.SALARY AND
       E.DNO=D.NUMBER AND D.MGRSSN=M.SSN))
```

Using General Assertions

- Specify a query that violates the condition; include inside a `NOT EXISTS` clause
- Query result must be empty
  - if the query result is not empty, the assertion has been violated

SQL Triggers

- Objective: to monitor a database and take action when a condition occurs
- Triggers are expressed in a syntax similar to assertions and include the following:
  - event (e.g., an update operation)
  - condition
  - action (to be taken when the condition is satisfied)

SQL Triggers: An Example

- A trigger to compare an employee's salary to his/her supervisor during insert or update operations:

```
CREATE TRIGGER INFORM_SUPERVISOR
BEFORE INSERT OR UPDATE OF
   SALARY, SUPERVISOR_SSN ON EMPLOYEE
   FOR EACH ROW
       WHEN
       (NEW.SALARY> (SELECT SALARY FROM EMPLOYEE
                     WHERE SSN=NEW.SUPERVISOR_SSN))
       INFORM_SUPERVISOR (NEW.SUPERVISOR_SSN,NEW.SSN;
```

Views in SQL

- A view is a “virtual” table that is derived from other tables
- Allows for limited update operations (since the table may not physically be stored)
- Allows full query operations
- A convenience for expressing certain operations

Specification of Views

- SQL command: `CREATE VIEW`
  - a table (view) name
  - a possible list of attribute names (for example, when arithmetic operations are specified or when we want the names to be different from the attributes in the base relations)
  - a query to specify the table contents

SQL Views: An Example

- Specify a different WORKS_ON table

```
CREATE VIEW WORKS_ON_NEW
AS SELECT FNAME, LNAME, PNAME, HOURS
     FROM EMPLOYEE, PROJECT, WORKS_ON
     WHERE SSN=ESSN AND PNO=PNUMBER
     GROUP BY PNAME;
```

Using a Virtual Table

- We can specify SQL queries on a newly create table (view):

```
SELECT FNAME, LNAME FROM WORKS_ON_NEW
WHERE PNAME='ProductX';
```

- When no longer needed, a view can be dropped:

```
DROP WORKS_ON_NEW;
```

Efficient View Implementation

- Query modification: present the view query in terms of a query on the underlying base tables
  - disadvantage: inefficient for views defined via complex queries (especially if additional queries are to be applied to the view within a short time period)
- View materialization: involves physically creating and keeping a temporary table
  - assumption: other queries on the view will follow
  - concerns: maintaining correspondence between the base table and the view when the base table is updated
  - strategy: incremental update

View Update

- Update on a single view without aggregate operations: update may map to an update on the underlying base table
- Views involving joins: an update *may* map to an update on the underlying base relations
  - not always possible
- Views defined using groups and aggregate functions are not updateable
- Views defined on multiple tables using joins are generally not updateable

Summary

- Additional features of SQL are discussed.
  - Nested queries, joined tables, aggregate functions, and grouping.
- We discussed the CREATE ASSERTION statement, which allows the specification of more general constraints on the database, and introduced the concept of triggers and the CREATE TRIGGER statement.
- Discussed SQL facility for defining views on the database.